

Dithered Belief Propagation for Real-Time Quantum Memory Decoding

Zhengyu Cai, Mohamad Ali Jarkas, Bokun Zhao, and Warren J. Gross

Department of Electrical & Computer Engineering

McGill University

Montreal, QC, Canada

{zhengyu.cai, mohamad.jarkas, bokun.zhao}@email.mcgill.ca, warren.gross@mcgill.ca

Abstract—We introduce dithered belief-propagation decoding for circuit-level quantum memory to mitigate the oscillations and trapping behavior that limit standard belief-propagation (BP) decoders for qLDPC codes. Our approach combines a quantum-adapted sign-flip mechanism with chained randomized prior perturbation. Evaluated under circuit-level noise on the distance-11 rotated surface code and the distance-12 bivariate-bicycle code, the proposed dithered decoder significantly outperforms BP and BP+OSD+CS-10. The proposed decoder achieves accuracy comparable to or better than state-of-the-art Relay-BP, especially at higher physical error rates, while trading a moderate increase in additions for about 98% fewer multiplications on average and eliminating node-level RNGs. These results suggest that dithering is a promising approach toward real-time decoding of quantum memories.

Index Terms—qLDPC codes, belief propagation decoding, quantum error correction

I. INTRODUCTION

Fault-tolerant quantum computation relies on quantum error correction to protect logical information against the noise affecting physical qubits, gates, measurements, and idle operations [1], [2]. A fundamental requirement for fault tolerance is quantum memory: the continuous active preservation of an encoded logical state over multiple error-correction cycles [3]. In practice, this protection requires a classical decoder that processes the syndrome data produced by repeated stabilizer measurements and outputs corrections quickly and accurately enough to keep pace with the quantum device [4].

Quantum LDPC codes have recently emerged as promising candidates for low-overhead fault tolerance [5], [6], but their decoding remains difficult [7]. Unlike simpler settings where some classical decoding ideas work well, qLDPC decoding graphs contain loops, symmetries, and degeneracies that cause standard belief propagation to oscillate, stall, and never converge, especially under circuit-level noise [8], [9].

Minimum-weight perfect matching (MWPM) [10], [11] is a natural reference point because it delivers high decoding accuracy for surface code [12] architectures and has very fast CPU implementations. Its strength lies in exploiting the pairing structure of detection events, making it especially effective for topological codes [13]–[15] whose decoding graphs exhibit a well-aligned geometric structure. The limitation however is that this advantage does not transfer cleanly to general qLDPC circuit decoding due to the absence of such structure [11], [16].

For general qLDPC codes, a major breakthrough was the introduction of BP+OSD, where ordered statistics decoding post-processes the output of a non-convergent belief-propagation run using reliability information and Gaussian elimination [17], [18]. This approach was historically important because it enabled successful decoding in much of the qLDPC landscape and became a widely used gold-standard baseline. Its weakness is computational cost: OSD depends on expensive matrix operations, especially Gaussian elimination, which make it slow even on CPUs and particularly unattractive for low-latency real-time implementations [19], [20]. Therefore, BP+OSD should be framed as a strong accuracy-oriented baseline, but one whose heavy post-processing motivates the search for decoders without needing Gaussian elimination while retaining comparable performance.

There had already been substantial effort to suppress trapping-set and oscillation pathologies while staying within a message-passing framework. Classical dithered BP [21] is an early example: it introduces randomized perturbations such as random sign flips and randomized initial states to escape BP failure modes, and was explicitly designed to reduce error floors with minimal hardware overhead rather than through OSD-like post-processing. In the qLDPC setting, work on enhanced Min-Sum [22] considered the oscillatory behavior of Min-Sum on symmetric stabilizers of two-block CSS codes and proposed an iteration-dynamics-memory variant in which past messages are reused and, when consecutive variable-to-check messages disagree in sign, the current and previous messages are combined to dampen the oscillation. They report large gains over normalized Min-Sum and even slight improvements over BP+OSD-0 [18], i.e., OSD with no combination sweep, on binary-symmetric-channel bivariate-bicycle benchmarks while retaining linear complexity. However, these efforts still did not establish a realistic circuit-level alternative to BP+OSD for general qLDPC decoding.

Recent work [23], [24] by IBM has shown that one does not need to rely on OSD-style post-processing to exceed BP+OSD-level decoding performance. Relay-BP demonstrates this by augmenting belief propagation with disordered memory strengths and ensembling, achieving substantially better performance than BP+OSD+CS-10 on bivariate-bicycle benchmarks [6]. As a result, Relay-BP provides a natural modern baseline for message-passing-based qLDPC quantum-memory

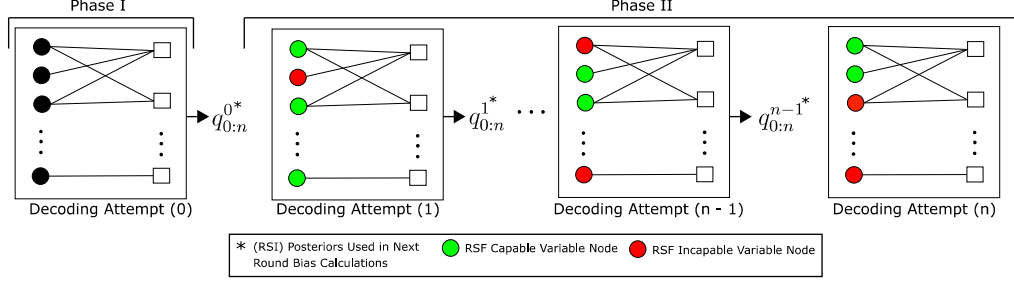


Fig. 1. The proposed qDither Tanner graph architecture.

decoding [25]. Instead of using a fixed prior term throughout the iterations, Relay-BP recomputes a memory-weighted bias at every timestep. This extra repeated per-iteration update makes Relay-BP structurally more involved and diverges from simple, hardware-friendly primitives that Min-Sum provides.

In this work, we introduce an enhanced quantum variant of the dithered Min-Sum decoder that conquers non-converging behavior of belief propagation while retaining the simplest message-passing primitives.

Our contributions are listed as follows:

- We propose qDither, a quantum dithered min-sum decoding algorithm that outperforms Relay-BP on surface code and bivariate-bicycle code (Gross code) at higher noise levels.
- We show that qDither eliminates 98% of the multiplications in Relay-BP at the cost of a moderate increase in additions, and requires minimal augmentation to the standard syndrome-based Min-Sum message-passing structure making it more suitable for real-time decoding implementation.

II. BACKGROUND

Quantum error correction protects logical information by repeatedly measuring check operators and using the resulting classical syndrome data to infer a correction. In the idealized picture, an unknown physical error acts on the encoded state, the measured syndrome identifies the violated checks, and a decoder returns a correction that aims to restore the state to the codespace [8], [26], [27].

A decoding instance is specified by a parity-check matrix and a logical observable matrix

$$H \in \mathbb{F}_2^{M \times N}, \quad A \in \mathbb{F}_2^{K \times N},$$

and a fault-probability vector

$$p = (p_1, \dots, p_N) \in [0, 1]^N.$$

Here, the N columns correspond to elementary error events, or fault locations, and the M rows correspond to detector outcomes. Let

$$E \in \mathbb{F}_2^N$$

denote the unknown error pattern, where $E_j = 1$ indicates that error event j occurred. Assuming independent error events, the probability of E is

$$\Pr(E) = \prod_{j=1}^N (1 - p_j) \left(\frac{p_j}{1 - p_j} \right)^{E_j}. \quad (1)$$

The observed syndrome is the binary vector

$$\sigma = HE \in \mathbb{F}_2^M, \quad (2)$$

where all arithmetic is over $\mathbb{F}_2$. The logical action induced by the error pattern is similarly represented by

$$a(E) = AE \in \mathbb{F}_2^K. \quad (3)$$

A decoder is a classical map

$$D : \mathbb{F}_2^M \rightarrow \mathbb{F}_2^N$$

that takes the syndrome σ as input and outputs an estimated correction

$$\hat{E} = D(\sigma).$$

The decoder should return a syndrome-consistent estimate, i.e.

$$H\hat{E} = \sigma. \quad (4)$$

However, the true objective is not exact recovery of E , since many distinct error patterns can produce the same syndrome. Instead, decoding succeeds when the residual error $E + \hat{E}$ has trivial logical action:

$$A(E + \hat{E}) = 0. \quad (5)$$

Equivalently, $\hat{E}$ and E must belong to the same logical coset.

This binary description induces a Tanner graph $G(H)$. The graph is bipartite: each column of H defines an variable node, each row defines a check node, and an edge connects variable node j to check node i whenever $H_{ij} = 1$. We write $N(i)$ for the set of variable nodes adjacent to check node i , and $N(j)$ for the set of check nodes adjacent to variable node j . In this graph-theoretic picture, the syndrome is the set of check nodes incident to an odd number of active variable nodes, and decoding amounts to finding a set of variable nodes whose boundary reproduces the observed syndrome.

A. Syndrome based Min-Sum Decoding

We next recall the syndrome-based Min-Sum decoder [28] for the binary quantum decoding problem. The decoder operates on the Tanner graph associated with the binary check matrix H , where each column of H corresponds to an variable node and each row corresponds to a check node. Let $N(i)$ denote the set of variable nodes adjacent to check node i , and let $N(j)$ denote the set of check nodes adjacent to variable node j .

For each error location j , we define the prior log-likelihood ratio

$$\lambda_j = \ln\left(\frac{1-p_j}{p_j}\right). \quad (6)$$

In standard quantum Min-Sum decoding, these prior terms remain fixed throughout the iterations.

The decoder iteratively exchanges real-valued messages along edges of the Tanner graph. We denote by $m_{j \rightarrow i}^{(\ell)}$ the message sent from variable node j to check node i at iteration ℓ , and by $m_{i \rightarrow j}^{(\ell)}$ the message sent in the reverse direction.

a) Initialization.: At the start of decoding, each variable-to-check message is initialized by the corresponding prior:

$$m_{j \rightarrow i}^{(0)} = \lambda_j. \quad (7)$$

b) Check-node update.: Given the incoming variable-to-check messages from the previous iteration, the check-to-variable update is

$$s_{i \rightarrow j}^{(\ell)} = (-1)^{\sigma_i} \prod_{j' \in N(i) \setminus \{j\}} \text{sgn}(m_{j' \rightarrow i}^{(\ell-1)}) \quad (8)$$

$$m_{i \rightarrow j}^{(\ell)} = s_{i \rightarrow j}^{(\ell)} \min_{j' \in N(i) \setminus \{j\}} |m_{j' \rightarrow i}^{(\ell-1)}| \quad (9)$$

c) Variable-node update.: Each variable node then combines its prior with the incoming check messages from all neighboring checks except the destination check:

$$m_{j \rightarrow i}^{(\ell)} = \lambda_j + \sum_{i' \in N(j) \setminus \{i\}} m_{i' \rightarrow j}^{(\ell)}. \quad (10)$$

d) Marginals and hard decisions.: After the edge updates at iteration ℓ , the marginal log-likelihood ratio at variable node j is computed as

$$L_j^{(\ell)} = \lambda_j + \sum_{i' \in N(j)} m_{i' \rightarrow j}^{(\ell)}. \quad (11)$$

Negative values of $L_j^{(\ell)}$ favor the presence of an error. Accordingly, the hard decision is

$$\hat{E}_j^{(\ell)} = \begin{cases} 1, & L_j^{(\ell)} < 0, \\ 0, & L_j^{(\ell)} \geq 0. \end{cases} \quad (12)$$

e) Stopping rule.: Let

$$\hat{E}^{(\ell)} = (\hat{E}_1^{(\ell)}, \hat{E}_2^{(\ell)}, \dots, \hat{E}_N^{(\ell)}) \in \mathbb{F}_2^N.$$

If the estimated error pattern satisfies

$$H\hat{E}^{(\ell)} = \sigma, \quad (13)$$

Then the decoder halts and outputs $\hat{E}^{(\ell)}$; otherwise, the iterations continue until a pre-defined maximum iteration budget is reached.

B. Classical Dithered Min-Sum Decoding

In a dithered decoder, in the event that the decoder fails to find a solution, a randomized second decoding phase (Phase-II) is invoked [21]. The two dithering mechanisms are:

a) Random sign flip (RSF): In this variant, the Min-Sum check-node update is modified by a random sign variable. For each check node i and iteration ℓ , let

$$a_i^{(\ell)} \in \{-1, +1\}, \quad \Pr(a_i^{(\ell)} = -1) = \rho,$$

with all draws independent. The check-to-variable message becomes

$$m_{i \rightarrow j}^{(\ell)} = a_i^{(\ell)} m_{i \rightarrow j}^{(\ell-1)} \quad (14)$$

b) Random initial state (RIS): In this variant, one performs ensembling multiple decoding trials with randomly perturbed inputs. For trial r , define a dither vector

$$d^{(r)} = (d_1^{(r)}, \dots, d_N^{(r)}),$$

whose components are i.i.d. with zero mean. The r -th trial then runs the ordinary Min-Sum recursion with initialization

$$\lambda_j^{(r)} = \lambda_j + d_j^{(r)} = m_{j \rightarrow i}^{(r,0)}. \quad (15)$$

III. QUANTUM DITHERED MIN-SUM DECODING (qDITHER)

In this quantum variant of dithered Min-Sum (qDither), as shown in Figure 1, we modify both RSF and RIS to better address degeneracy-induced oscillations.

a) Enhanced RSF: We build upon the soft sign-flip rule proposed by Chytas *et al.* [22]. Utilizing the symmetric structure of CSS stabilizers, their method relies on separating the XZ decoding problem into two identical sub-decoding problems at code-level. We extend this approach into a stochastic variant to serve as a true dithering mechanism for general qLDPC codes at circuit-level. Instead of randomly flipping check-node signs like classical RSF, we first randomly partition all variable nodes into two groups. The first group performs the standard variable node update, while the second group detect sign flips between the current and previous message and merges them.

$$\delta_{j \rightarrow i}^{(\ell)} = \frac{1 - \text{sgn}(m_{j \rightarrow i}^{(\ell)}) \text{sgn}(m_{j \rightarrow i}^{(\ell-1)})}{2} \quad (16)$$

$$m_{j \rightarrow i}^{(\ell)} = \delta_{j \rightarrow i}^{(\ell)} m_{j \rightarrow i}^{(\ell-1)} + m_{j \rightarrow i}^{(\ell)} \quad (17)$$

The probability ρ that a variable node receives this mechanism is a hyperparameter. Larger ρ increases the influence of the soft sign-flip correction rule, which may stabilize oscillatory message dynamics and lead to faster convergence, but can also reduce exploratory diversity. This mechanism may also be enabled in Phase I.

b) Enhanced RIS: Instead of ensembling independent trials, Phase-II now employs a chained design in which the decoder inherits the posterior produced by the preceding decoder, to form a new input bias by blending it with the fixed base bias.

For each variable node j , the Phase-II bias for the r th decoder is defined as

$$\lambda_j^{(r)} = (1 - \chi_j^{(r)})\lambda_j^{\text{base}} + \chi_j^{(r)}q_j^{(r-1)}, \quad \chi_j^{(r)} \sim \text{Unif}[\alpha, \beta] \quad (18)$$

where $q^{(r-1)}$ is the posterior of $(r-1)$ th decoder.

The parameters α and β bound the random mixing strength χ . In our experiments, we use $\beta \leq 1.0$ and $\alpha \geq -0.15$ as conservative settings, so that inherited posterior information influences the base prior, while still allowing occasional negative correction to escape an unfavorable inherited direction.

The pseudocode of the proposed qDither procedure is summarized in Algorithm 1.

IV. RESULTS

We benchmark the proposed decoder under a circuit-level noise model [5], [12] using the same evaluation protocol adopted in prior circuit-based BP studies [7], [23]–[25]. Each benchmark instance is generated by repeating noisy measurements and appending a final noiseless round. Under the circuit-level noise model, each elementary circuit location fails independently according to its corresponding fault channel, and each individual fault mechanism is treated as a candidate error event in the decoding problem [29], [30].

These fault events induce a decoding object specified by a circuit-level parity-check matrix H , a logical observable matrix A , and a prior probability vector p . The matrix H records which detector outcomes are flipped by each fault event, while A records whether those faults flip the logical observables of the code. The decoder is then supplied only with the observed detector vector and must output a candidate correction consistent with this induced decoding instance.

To reduce redundancy, fault events with identical detector signatures may be compressed into a single effective variable with a new prior. Decoder performance is evaluated by Monte Carlo simulation over a range of physical noise rates. Both of our decoding examples used are CSS-type [31]. We will evaluate primarily the logical error rate (LER) and secondly, the iteration count. The hyperparameters are chosen and optimized through sweeping over 1000 samples at noise level $p = 0.005$.

A. Surface Code

We consider the circuit-level decoding problem generated by the distance-11 rotated surface code [32]. We derive separate H_X and H_Z decoding instances through basis filtering

Algorithm 1 Quantum Dithered Min-Sum

Input: $H, \sigma, p, n, b, \alpha, \beta, \rho$

Output: $\hat{E}$

Phase I:

1: Compute λ_j^{base} from p // Eq. (6)

2: $m_{j \rightarrow i}^{(0)} \leftarrow \lambda_j^{\text{base}}$ // Eq. (7)

3: Compute $\hat{E}$ via standard Min-Sum // Eqs. (8)–(11)

4: **if** $H\hat{E} = \sigma$ **then**

5: **return** $\hat{E}$

6: **end if**

Phase II:

7: $q_j^{(0)} \leftarrow \lambda_j^{\text{base}}$

8: **for** $r = 1 \dots n$ **do**

9: Draw $\chi_j^{(r)} \sim \text{Unif}[\alpha, \beta]$

10: Compute $\lambda_j^{(r)}$ from $\lambda_j^{\text{base}}, \chi_j^{(r)}, q_j^{(r-1)}$ // Eq. (18)

11: $m_{j \rightarrow i}^{(0)} \leftarrow \lambda_j^{(r)}$

12: Draw $I_j^{(r)} \sim \text{Bernoulli}(\rho)$

13: **for** $l = 1 \dots b$ **do**

14: Compute $m_{i \rightarrow j}^{(l)}$ // Eq. (9)

15: Compute $m_{j \rightarrow i}^{(l)}$ // Eq. (10)

16: **if** $I_j^{(r)} == 1$ **then**

17: **if** $\text{sgn}(m_{j \rightarrow i}^{(l)}) \neq \text{sgn}(m_{j \rightarrow i}^{(l-1)})$ **then**

18: $m_{j \rightarrow i}^{(l)} \leftarrow m_{j \rightarrow i}^{(l-1)}$

19: **end if**

20: **end if**

21: Compute $L_j^{(l)}$ from $\lambda_j^{(r)}, m_{i \rightarrow j}^{(l)}$ // Eq. (11)

22: Compute $\hat{E}^{(l)}$ from $L_j^{(l)}$ // Eq. (12)

23: **if** $H\hat{E}^{(l)} = \sigma$ **then**

24: **return** $\hat{E}^{(l)}$

25: **end if**

26: **end for**

27: $q_j^{(r)} \leftarrow L_j^{(l)}$

28: **end for**

29: **return** $\hat{E}^{(l)}$

and perform XZ decoding. We use 3000 decoder chains, draw the dither as $\chi \sim \text{Unif}[-0.14, 0.85]$, and limit each decoder call to at most $b = 15$ iterations. In addition, we allow the decoder to collect up to five distinct solutions and return the one with the lowest weighted LLR.

Figure 2 shows the logical error rate per cycle—equivalently, per round in this setting—for varying noise levels and different decoders. For $p \geq 0.003$, qDither-BP-5 outperforms Relay-BP-5 [23]. Although MWPM [11] still achieves the lowest LER, qDither-BP surpasses the remaining message-passing decoders by orders of magnitude. Mem-BP [23], [33], [34] introduces a memory term that updates a local bias every iteration. Relay-BP builds upon Mem-BP by randomizing the memory strength across relay legs and ensembling decoders by passing forward the final marginals. Structurally, qDither-BP is a low-overhead perturbative extension of standard Min-Sum-BP, whereas Relay-BP is a stateful multi-leg decoder that

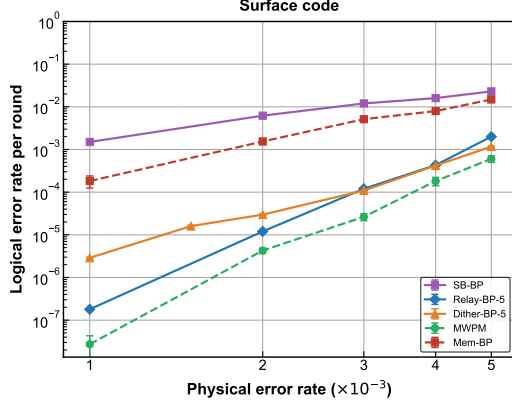


Fig. 2. LER of decoding distance-11 rotated surface code, compared with syndrome-based Min-Sum (SB-BP), Relay-BP, Mem-BP and MWPM.

augments BP with dynamic memory updates.

B. Gross code

Next, we test our decoder on the $[[144, 12, 12]]$ bivariate-bicycle code, also known as the Gross code [5]. The full parity-check matrix has shape 1872×71280 . In this experiment, we examine both XZ and XYZ decoding, with RSF enabled for both phases. In XYZ decoding, we pass the full parity-check matrix directly into the decoder, thereby preserving correlations between errors. In the memory benchmark, 12 Bell pairs are prepared, with one half of each pair fed into the Gross-code encoder while the other halves remain as noiseless references. Here we use 1500 decoder chains, a decoding budget of $b = 100$, a dither range $\chi \sim \text{Unif}[-0.07, 0.5]$, and $\rho = 0.65$.

Figure 3 evaluates our decoders on the Gross code. For independent XZ decoding, we benchmark against syndrome-based Min-Sum, Relay-BP-5 and BP+OSD+CS-10 [18] that allows OSD to use up to 10 combination sweeps for syndrome matching. The overall trend is that plain syndrome-based Min-Sum remains the weakest method, surpassed by other models by a huge margin. Notably, qDither-BP-5 demonstrates accuracy nearly on par with Relay-BP-5, and outperforms BP+OSD+CS-10 at all noise levels.

For XYZ decoding, we compare XYZ-qDither-1 with XYZ-Relay-1. At $p = 3 \times 10^{-3}$, the proposed XYZ-qDither-BP-1 achieves a logical error rate of approximately $(7.15 \pm 0.5) \times 10^{-6}$, which is very competitive with the $(7 \pm 1) \times 10^{-6}$ reported for XYZ-Relay-1. At $p = 5 \times 10^{-3}$, notably, qDither-BP outperforms Relay-BP, suggesting that dithering provides a stronger advantage in the higher-noise regime. This noise range reflects the noise levels of near-term superconducting quantum architectures [35]–[37], and it aligns exactly with the benchmarking regime established in prior works [23], [24], [33]. Taken together, these results indicate that qDither-BP attains Relay-level performance at low physical error rates while offering improved robustness at larger p .

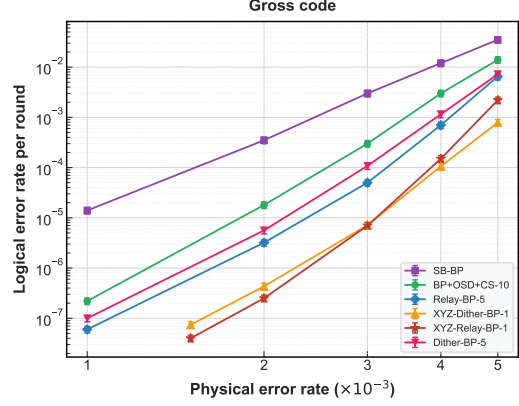


Fig. 3. LER of decoding Gross code, compared with syndrome-based Min-Sum (SB-BP), BP+OSD-CS-10, Relay-BP-5, qDither-BP-5 with XZ decoding, as well as XYZ-Relay-BP-1 and XYZ-qDither-BP-1.

V. REAL-TIME DECODING

A. Algorithmic Complexity

To achieve real-time decoding, the decoder must produce results faster than the generation of a QEC round to avoid the buffer backlog issue [38]. For the distance-12 bivariate-bicycle code, using a $1 \mu\text{s}$ QEC cycle time [4] would allow a temporal budget of $12 \mu\text{s}$ per window. In order to meet this timing requirement, Müller *et al.* [23] suggest that an unrolled design with a critical path that allows for a clock period of 20 ns (per iteration) would translate to a ~ 600 BP iteration budget. Consequently, a decoder is only practically viable in this regime if it achieves low logical error rates while keeping its average iteration count within this effective real-time budget, assuming a comparable per-iteration workload.

In this work, we consider algorithmic complexity from two complementary perspectives, namely iteration and operation complexity. The former measures the average number of BP iterations needed to perform one decoding task; the latter accounts for the number of operations within each BP iteration.

1) *Iteration Complexity*: Figure 4 shows a clear tradeoff between decoder cost and accuracy as the number of chains increases at $p = 0.003$. For both XZ and XYZ decoding with 1 solution sought, increasing the chain count steadily lowers the logical error rate, with the largest gains occurring at small to intermediate chain counts. With more chains, however, the improvement in logical error rate becomes more gradual and the average iteration count appears to cap under 250 iterations per shot despite having more available budget. This saturation is particularly noticeable for XYZ-qDither. This shows that even though we use a high number of chained decoders, qDither manages to decode well under the required iteration budget.

Figure 5 shows how the average decoding cost changes as the RSF parameter ρ is varied at $p = 0.003$. When RSF is disabled, i.e., standard Syndrome-based Min-Sum, the decoder requires a large number of BP iterations per shot, making it hard to satisfy a realistic real-time decoding budget; this

effect is especially pronounced for XYZ decoding. As ρ is increased from zero, the average iteration count drops sharply. Interestingly, the cost rises again near $\rho = 1$, suggesting that this extra degree of perturbation helps dampen oscillation and our proposed RSF mechanism provides substantial speedup in iterations needed to converge.

2) *Operation Complexity*: Let ξ, V, N, t be the number of edges, variable nodes, average chains and iterations respectively. A variable node gathers all incoming messages and then subtracts exactly one of them for each outgoing message; this corresponds to a total of 2ξ number of additions. Internal RSF yields approximately $\rho\xi$ extra additions. Relay-BP on the other hand, requires $2\xi + V$ additions and $2V$ multiplications for node-wise bias update.

To set up each Phase-II RIS, one computes new priors which demands a total of $2(N - 1)V$ multiplications and $(N - 1)V$ additions. The XYZ parity-check matrix for the Gross code has shape $1872 \times 71,280$, with $\xi = 419,904$. At noise level $p = 0.003$, XYZ-qDither-1 has $V = 71,280$, $N = 2.6$, and $t = 200$. This yields a setup cost of 228,096 multiplications and 114,048 additions, as well as a computation cost of 530ξ , for a total of 222,663,168 additions. We implemented XYZ-Relay-1 using the best parameters reported. This model reports $t = 85$, yielding a total cost of 77,442,480 additions and 12,117,600 multiplications.

These figures indicate that an unrolled hardware implementations of Relay-BP would be substantially larger. Although qDither requires approximately $2.88\times$ more additions, it uses only 2% of the multiplications required by Relay-BP. This gives qDither a clear advantage due to the complete absence of multiplication during effective decoding stage.

B. Hardware Mapping

Valls *et al.* [19] suggest an unrolled architecture for VNUs and CNUs to achieve the latency required to avoid the backlog issue. Recently, Maurer *et al.* [24] implemented a hardware-friendly approximate XZ-Relay-BP decoder on an AMD VU19P FPGA. They replaced floating point multipliers by bit-shifters and adders, with quantization down to 4 bits and limiting memory factors to a predetermined range. This allows them to expand BP in a fully unrolled fashion to meet the real-time constraint at the cost of slightly higher LER. However, the internal state update mechanism enforces that each Variable Node Unit (VNU) must carry its own RNG, as well as required auxiliary logic to blend the bias (which consists of multipliers and adders). As a result, their VNUs collectively take up 67.2% of LUTs. For full XYZ decoding however, there are substantially more (71280) variable nodes than split X/Z (8784), and XYZ-Relay-BP requires at least 6 bits of precision to retain its LER [24]. This makes an unrolled XYZ-Relay-BP architecture, similar to that implemented for the XZ-Relay-BP, challenging to fit onto a modern FPGA without significant modifications.

Because qDither's RSF design eliminates the need for per-iteration bias blending, we can replace the RNG and auxiliary logic with simpler oscillation detection hardware. The simplest

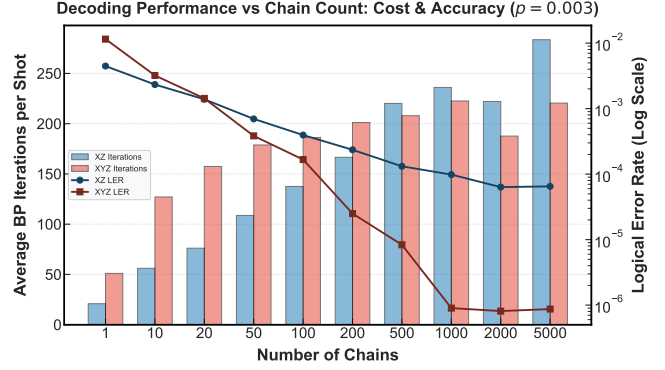


Fig. 4. Average BP iteration per shot for decoding Gross code with varying number of chains and respective LER at noise level $p = 0.003$.

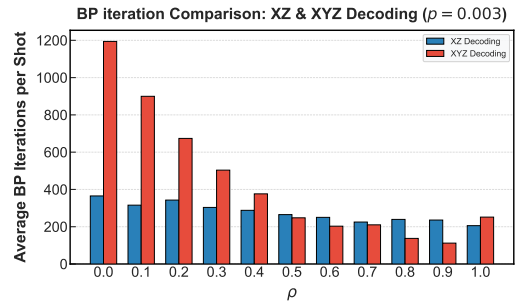


Fig. 5. Average BP iteration per shot for decoding Gross code under varying RSF ρ at noise level $p = 0.003$.

implementation can include an XOR gate between the most significant register bits of the previous and current marginals, a multiplexer, and an adder. Furthermore, RIS blending only occurs during posterior probability hand-off between attempts and requires additional logic at the decoder level. This permits a much lighter VNU design, making qDither a viable candidate for scaling to full XYZ decoding.

VI. CONCLUSION

In this work, we showed that the internal state evolution from Relay-BP is not a necessary condition for belief propagation to escape non-converging behaviors. RIS allows the decoder to start its additional attempt from an updated prior, and RSF enables oscillation dampening and faster convergence. Based on our experiments, we achieve near or better than Relay-BP performance on surface and bivariate-bicycle codes with projected less hardware overhead. qDither trades a moderate increase in additions for a reduction in area through the elimination of node-level RNG and a 98% reduction in multiplications. Our future directions include hardware architectures, efficient hyperparameter tuning, and benchmarking on additional code families.

REFERENCES

- [1] A. M. Steane, "Error correcting codes in quantum theory," *Physical Review Letters*, vol. 77, no. 5, p. 793, 1996.

- [2] A. Hashim, L. B. Nguyen, N. Goss, B. Marinelli, R. K. Naik, T. Chistolini, J. Hines, J. P. Marceaux, Y. Kim, P. Gokhale *et al.*, “Practical introduction to benchmarking and characterization of quantum computers,” *PRX Quantum*, vol. 6, no. 3, p. 030202, 2025.
- [3] X. Yuan, Y. Liu, Q. Zhao, B. Regula, J. Thompson, and M. Gu, “Universal and operational benchmarking of quantum memories,” *npj Quantum Information*, vol. 7, no. 1, p. 108, 2021.
- [4] F. Battistel, C. Chamberland, K. Johar, R. W. Overwater, F. Sebastiano, L. Skoric, Y. Ueno, and M. Usman, “Real-time decoding for fault-tolerant quantum computing: Progress, challenges and outlook,” *Nano Futures*, vol. 7, no. 3, p. 032003, 2023.
- [5] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, “High-threshold and low-overhead fault-tolerant quantum memory,” *Nature*, vol. 627, no. 8005, pp. 778–782, 2024.
- [6] A. A. Kovalev and L. P. Pryadko, “Quantum kronecker sum-product low-density parity-check codes with finite rate,” *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 88, no. 1, p. 012311, 2013.
- [7] A. S. Maan, F. M. Garcia Herrero, A. Paler, and V. Savin, “Decoding correlated errors in quantum ldpc codes,” *Nature Communications*, 2026.
- [8] K.-Y. Kuo and C.-Y. Lai, “Exploiting degeneracy in belief propagation decoding of quantum codes,” *npj Quantum Information*, vol. 8, no. 1, p. 111, 2022.
- [9] M. Moradi, V. Nourozi, S. Habib, and D. G. Mitchell, “Learning to decode quantum ldpc codes via belief propagation,” *arXiv preprint arXiv:2603.10192*, 2026.
- [10] J. Edmonds, “Paths, trees, and flowers,” *Canadian Journal of mathematics*, vol. 17, pp. 449–467, 1965.
- [11] O. Higgott, “Pymatching: A python package for decoding quantum codes with minimum-weight perfect matching,” *ACM Transactions on Quantum Computing*, vol. 3, no. 3, pp. 1–16, 2022.
- [12] S. B. Bravyi and A. Y. Kitaev, “Quantum codes on a lattice with boundary,” *arXiv preprint quant-ph/9811052*, 1998.
- [13] N. P. Breuckmann and B. M. Terhal, “Constructions and noise threshold of hyperbolic surface codes,” *IEEE transactions on Information Theory*, vol. 62, no. 6, pp. 3731–3744, 2016.
- [14] J. P. Bonilla Ataides, D. K. Tuckett, S. D. Bartlett, S. T. Flammia, and B. J. Brown, “The xxxz surface code,” *Nature communications*, vol. 12, no. 1, p. 2172, 2021.
- [15] M. Li, D. Miller, M. Newman, Y. Wu, and K. R. Brown, “2d compass codes,” *Physical Review X*, vol. 9, no. 2, p. 021041, 2019.
- [16] S. Miao, A. Schnerring, H. Li, and L. Schmalen, “Quaternary neural belief propagation decoding of quantum ldpc codes with overcomplete check matrices,” *IEEE Access*, vol. 13, pp. 25 637–25 649, 2025.
- [17] P. Panteleev and G. Kalachev, “Degenerate quantum ldpc codes with good finite length performance,” *Quantum*, vol. 5, p. 585, 2021.
- [18] J. Roffe, D. R. White, S. Burton, and E. Campbell, “Decoding across the quantum low-density parity-check code landscape,” *Physical Review Research*, vol. 2, no. 4, p. 043423, 2020.
- [19] J. Valls, F. Garcia-Herrero, N. Raveendran, and B. Vasić, “Syndrome-based min-sum vs osd-0 decoders: Fpga implementation and analysis for quantum ldpc codes,” *IEEE Access*, vol. 9, pp. 138 734–138 743, 2021.
- [20] W. Fan, Y. Suzuki, G. S. Ravi, Y. Ueno, K. Inoue, and T. Tanimoto, “Accelerating bp-osd decoder for qldpc codes with local syndrome-based preprocessing,” *arXiv preprint arXiv:2509.01892*, 2025.
- [21] F. Leduc-Primeau, S. Hemati, S. Mannor, and W. J. Gross, “Dithered belief propagation decoding,” *IEEE Transactions on Communications*, vol. 60, no. 8, pp. 2042–2047, 2012.
- [22] D. Chytras, N. Raveendran, and B. Vasic, “Enhanced min-sum decoding of quantum codes using previous iteration dynamics,” *arXiv preprint arXiv:2501.05021*, 2025.
- [23] T. Müller, T. Alexander, M. E. Beverland, M. Bühler, B. R. Johnson, T. Maurer, and D. Vandeth, “Improved belief propagation is sufficient for real-time decoding of quantum memory,” *arXiv preprint arXiv:2506.01779*, 2025.
- [24] T. Maurer, M. Bühler, M. Kröner, F. Haverkamp, T. Müller, D. Vandeth, and B. R. Johnson, “Real-time decoding of the gross code memory with fpgas,” *arXiv preprint arXiv:2510.21600*, 2025.
- [25] S. Maurya, T. Maurer, M. Bühler, D. Vandeth, and M. E. Beverland, “Fpga-tailored algorithms for real-time decoding of quantum ldpc codes,” *arXiv preprint arXiv:2511.21660*, 2025.
- [26] A. Kissinger and J. van de Wetering, *Picturing Quantum Software: An Introduction to the ZX-Calculus and Quantum Compilation*. Preprint, 2024.
- [27] S. J. Devitt, W. J. Munro, and K. Nemoto, “Quantum error correction for beginners,” *Reports on Progress in Physics*, vol. 76, no. 7, p. 076001, 2013.
- [28] M. P. Fossorier, M. Mihaljevic, and H. Imai, “Reduced complexity iterative decoding of low-density parity check codes based on belief propagation,” *IEEE Transactions on communications*, vol. 47, no. 5, pp. 673–680, 1999.
- [29] M. A. Perlin, “qLDPC,” <https://github.com/qLDPCOrg/qLDPC>, 2023.
- [30] C. Gidney, “Stim: a fast stabilizer circuit simulator,” *Quantum*, vol. 5, p. 497, 2021.
- [31] A. R. Calderbank and P. W. Shor, “Good quantum error-correcting codes exist,” *Physical Review A*, vol. 54, no. 2, p. 1098, 1996.
- [32] D. Horsman, A. G. Fowler, S. Devitt, and R. V. Meter, “Surface code quantum computing by lattice surgery,” *New Journal of Physics*, vol. 14, no. 12, p. 123011, 2012.
- [33] J. Chen, Z. Yi, Z. Liang *et al.*, “Improved belief propagation decoding algorithms for surface codes,” *IEEE Transactions on Quantum Engineering*, 2025.
- [34] E. Nachmani, E. Marciano, L. Lugosch *et al.*, “Deep learning methods for improved decoding of linear codes,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 12, no. 1, pp. 119–131, 2018.
- [35] T. J. Maldonado, J. Flick, S. Krastanov, and A. Galda, “Error rate reduction of single-qubit gates via noise-aware decomposition into native gates,” *Scientific Reports*, vol. 12, no. 1, p. 6379, 2022.
- [36] K. Georgopoulos, C. Emary, and P. Zuliani, “Modeling and simulating the noisy behavior of near-term quantum computers,” *Physical Review A*, vol. 104, no. 6, p. 062432, 2021.
- [37] “Suppressing quantum errors by scaling a surface code logical qubit,” *Nature*, vol. 614, no. 7949, pp. 676–681, 2023.
- [38] B. M. Terhal, “Quantum error correction for quantum memories,” *Reviews of Modern Physics*, vol. 87, no. 2, p. 307–346, Apr. 2015. [Online]. Available: <http://dx.doi.org/10.1103/RevModPhys.87.307>